

Homework 5

STATISTICAL MODELING AND LEARNING

due: **Friday, March 13, 2026 at 10:00a**

Instructions: Please submit solutions on Populi as a knitted markdown pdf. For handwritten work, please scan your solution pages and include in the final pdf. Homeworks can be completed individually or in groups of up to three people. Names of all group members must be included on the writeups.

Problem 1: Neural Networks.

Part (a): Gradient calculation and gradient descent (calculus).

Consider a single neuron with one input: $\hat{y} = \sigma(wx + b)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid. For one data point (x, y) , the MSE loss is $L = \frac{1}{2}(\hat{y} - y)^2$.

- Let $z = wx + b$. Using the chain rule, derive $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$ in terms of $\hat{y}$, y , x , and $\sigma'(z)$. (Recall: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.)
- For $(x, y) = (2, 1)$ and $(w, b) = (0.5, 0)$: compute z , $\hat{y}$, L , and the gradients $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$.
- Perform one gradient descent step with learning rate $\alpha = 0.5$. Report the new (w, b) and new $\hat{y}$. Did $\hat{y}$ move toward $y = 1$?
- In **R** or **Python**: Implement gradient descent for this sigmoid neuron. Use data $(x, y) = (2, 1)$, initial $(w, b) = (0.5, 0)$, $\alpha = 0.5$, and run 20 iterations. Report the final (w, b) and plot L versus iteration.

Part (b): Image classification with a neural network.

In this problem, you will train a neural network to classify satellite images. Use a **smaller subset** of the EuroSAT dataset (land-use classification) to keep runtimes manageable. Load the data via PyTorch's `torchvision.datasets.EuroSAT`, which downloads automatically. For a quicker run, use only 3–4 classes and limit to ~ 200 images per class for training.

Use an 80/20 train–test split. Summarize your model and its accuracy. Include **at a minimum**:

- Overall test-set accuracy (proportion correctly classified).
- A grid of example images from the test set with your model's predicted classes (and actual classes).
- A confusion matrix: a table cross-tabulating (actual class, predicted class) on the test set.

Use PyTorch in a Jupyter notebook with `torchvision.datasets.EuroSAT`. Starter code is in `StatisticalModeling/code/HW5_neural_networks.ipynb`. You may use a simple CNN or a feed-forward network on flattened images; the goal is to understand the pipeline.

Problem 2: Difference-in-Differences.

This problem analyzes the effect of castle-doctrine (“stand your ground”) laws on violent crime. These laws expand legal justifications for using lethal force in self-defense. Load the data from the `causaldata` package: `data(castle, package = "causaldata")`. The data come from Cheng and Hoekstra (2013), combining FBI Uniform Crime Reports with information on state adoption of castle-doctrine laws (2000–2010).

Create a `treated` variable: `treated = 1` if the state ever adopted a castle-doctrine law during the sample period (i.e., `post` equals 1 for at least one year in that state), else `treated = 0`. For the regression and DiD, define a common “post” period: `post_did = 1` if (`treated = 1` and `post = 1`) or (`treated = 0` and `year ≥ 2007`), else `post_did = 0`. This ensures control states have a “late” period (2007–2010) comparable to treated states’ post-adoption years. Use `murder` (murder rate per 100,000) or `l_homicide` (logged homicide rate) as the outcome Y_{it} .

- a. Compute the four group means: (i) treated states, before adoption (`post_did = 0`); (ii) treated states, after adoption (`post_did = 1`); (iii) control states, early period (`post_did = 0`); (iv) control states, late period (`post_did = 1`). Use the murder rate (or logged homicide) as the outcome.
- b. Compute the simple “before–after” difference in treated states. Compute the same in control states (early vs. late period). What is the difference-in-differences (DiD) estimate? Interpret: what does this number represent?
- c. Run a regression: $Y_{it} = \beta_0 + \beta_1 \cdot \text{Treated}_i + \beta_2 \cdot \text{Post}_{it} + \tau \cdot (\text{Treated}_i \times \text{Post}_{it}) + \varepsilon_{it}$, where `Post` is `post_did`. Report the coefficient $\hat{\tau}$ on the interaction. How does it compare to your DiD from part (b)?
- d. In one or two sentences, state the key assumption that DiD relies on. Why might non-adopting states be a reasonable control for adopting states in this setting?